

Détection d'anomalies dans des pièces industrielles

Adriano De La Baume

Vincent Vila

27 juin 2026

Chapitre 1

Objectifs du projet

1.1 Premier niveau

L'objectif de ce projet est de développer un modèle capable de détecter une anomalie sur une pièce industrielle à l'aide d'une image. Différentes catégories de pièces industrielles sont proposées. On utilisera un ensemble de photos sans défaut pour entraîner le modèle sur une catégorie à la fois. Le modèle devrait être capable d'identifier si une nouvelle photo est conforme au modèle (donc sans défaut) ou non conforme et contient donc des défauts. Il faudra être capable d'identifier où se situe le défaut s'il y en a un.

1.2 Second niveau

Un second objectif est d'identifier l'anomalie détectée, transformant ainsi le problème en classification multi-classes. Chaque pièce est associée à une série d'anomalies qui lui sont propres. Pour cette seconde partie, on devra être capable de classer automatiquement les anomalies qu'on détecte sur les pièces.

Chapitre 2

Jeu de données

Nous utiliserons les deux jeux de données suivants.

Dans les deux cas, les jeux de données sont constitués d'un ensemble de photos montrant des pièces industrielles avec ou sans défaut ou objets étrangers.

La structure des dossiers donne des informations sur les images, mais il n'y a pas de données additionnelles accompagnant ces dossiers et fichiers.

2.1 MVTec AD

« MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection », sous licence **CC BY-NC-SA 4.0** (BERGMANN et al. 2019)

Il peut être récupéré à cette URL : <https://www.mvtec.com/research-teaching/datasets/mvtec-ad>

Ou celle-ci : <https://www.kaggle.com/datasets/ipythonx/mvtec-ad/data>

2.1.1 Structure des données

Ces photos montrent des pièces différentes qui peuvent avoir ou non des anomalies.

Ces photos sont classées dans un premier niveau de dossiers qui correspond aux catégories de pièces. Un second niveau découpe le jeu de données en sous-dossiers :

- **train** : contient un ensemble d'images de pièces sans défaut
- **test** : contient des sous-dossiers avec un ensemble d'images de pièces avec et sans défaut
- et **ground_truth** : contient les mêmes sous-dossiers que **test** avec des images des masques surlignant les anomalies des pièces.

Dans les dossiers **test** et **ground_truth**, il y a un sous-dossier par type d'anomalie et un sous-dossier **good** pour les pièces sans défaut. Chaque image de pièce avec anomalie a un masque correspondant dans le dossier **ground_truth**.

2.2 RAD

RAD : « RAD : A Comprehensive Dataset for Benchmarking the Robustness of Image Anomaly Detection », sous licence **CC BY 4.0** (CHENG et al. 2024)

Il peut être récupéré à cette URL : <https://github.com/hustCYQ/RAD-dataset>

2.2.1 Structure des données

Ces photos montrent toutes une plaque métallique percée de trous mais il peut y avoir un objet étranger (qu'il faudra identifier) posé sur la plaque.

Les photos sont classées dans un premier niveau de dossiers qui correspond aux catégories d'objets étrangers. Un second niveau découpe le jeu de données en sous-dossiers :

- **train/good** : contient toujours le même ensemble d'images de pièces sans objet étranger
- **test/good** : contient toujours le même ensemble d'images de pièces sans objet étranger
- **test/defect** : contient un ensemble d'images de pièces avec un objet étranger
- et **ground_truth/defect** : contient des images des masques surlignant les objets étrangers dans les photos du dossier **test/defect**.

Chapitre 3

Analyse des données

3.1 Extraction des meta-données

Les jeux de données n'étant constitués que de photos dans des dossiers, nous avons développé un programme qui va lire tous ces dossiers et leur contenu et récupérer les metadonnées des images pour alimenter un CSV qui agrège toutes les informations.

Ce script est légèrement différent pour chaque jeu de données :

— **MVTec** : `list_images_project.py`

— **RAD** : `list_images_project_rad.py`

Il crée un fichier CSV avec une ligne par photo, avec les colonnes suivantes :

1. **category** : pièce photographiée, ou pour le RAD, objet étranger déposé sur la pièce
2. **type** : les jeux de données sont déjà découpés en *train* et *test*
3. **quality** : `good` s'il n'y pas de défaut, le type de défaut sinon
4. **file** : nom du fichier photo, sans son extension
5. **extension** : extension du nom du fichier photo
6. **width** : largeur de la photo, en pixels
7. **height** : hauteur de la photo, en pixels
8. **mode** : mode de la photo - RGB pour la couleur ou L pour niveau de gris
9. **bands** : nombre de dimensions pour chaque pixel (3 pour la couleur ou 1 pour le gris)
10. **type_image** : C pour couleur, G pour niveau de gris
11. **mask** : indique si une image de mask est disponible (pour montrer le défaut de l'image)
12. **mask_bands** : nombre de dimensions pour chaque pixel du masque
13. **mask_mode** : mode du masque - RGB pour la couleur ou L pour niveau de gris
14. **mask_type** : pour le masque, C pour couleur, G pour niveau de gris

3.2 Nettoyage des meta-données

On a créé le script `clean_metadata.csv` qui récupère les 2 CSV contenant les meta-données des deux jeux de données.

3.2.1 Cas particuliers du jeu de données RAD

Le jeu de données RAD contient 7 fichiers qui sont des copies d'autres images déjà présentes, avec le même nom suivi de "(1)". Ce sont les seuls noms de fichiers avec des parenthèses. On a donc supprimé les noms de fichiers qui contiennent une parenthèse.

Les images qui sont dans les dossiers "`train/good`" et "`test/good`" sont les mêmes dans tous les dossiers parents : `bolt`, `ribbon`, `sponge`, `tape`. On supprime donc tous les doublons pour ne garder que les images du dossier `bolt`.

La colonne **category** contient le nom du premier niveau de dossier, c'est-à-dire le nom de l'objet étranger qui est déposé sur la pièce. On va donc plutôt utiliser ce nom dans la colonne **quality** (sauf quand il n'y a pas de défaut et que la quality est "good").

On remplace ensuite le contenu de la colonne **category** par "`metal_plate`" puisque c'est une plaque métallique perforée qui est contrôlée sur tout le jeu de données RAD.

3.2.2 Concaténation des meta-données des deux jeux de données

On ajoute ensuite l'origine des données sur chaque photo pour pouvoir distinguer facilement les données "RAD" et "MVTec" puis on concatène les deux jeux de données dans un unique tableau.

3.2.3 Suppression des colonnes inutiles

Toutes les photos ont l'extension ".png" donc on peut supprimer cette colonne qui n'apporte aucune information.

Les masques n'ont toujours qu'une seule bande, avec le mode "L" et le type "G". On supprime donc aussi ces informations concernant les masques.

Les colonnes `mode`, `bands` et `type_image` sont redondantes. Quand on a 3 bandes, le mode est "RGB" et le type est "C" (couleur). Quand on a une bande, le mode est "L" et le type est "G" (niveau de gris). On supprime donc les colonnes `mode` et `type_image` pour ne garder que `bands`.

La colonne `mask` est également redondante avec la `quality`. Si la `quality` est "good", on n'a pas de défaut et donc pas de masque (`mask = False`), sinon, on a toujours un masque montrant le défaut (`mask = True`). On supprime donc la colonne `mask`.

3.2.4 Jeu de données fusionné

On obtient finalement un nouveau tableau (qu'on sauvegarde dans un nouveau CSV), avec ces colonnes :

1. `category` : pièce à analyser
2. `type` : les jeux de données sont déjà découpés en *train* et *test*
3. `quality` : good s'il n'y pas de défaut, le type de défaut (ou d'objet étranger) sinon
4. `file` : nom du fichier photo, sans son extension
5. `width` : largeur de la photo, en pixels
6. `height` : hauteur de la photo, en pixels
7. `bands` : nombre de dimensions pour chaque pixel (3 pour la couleur ou 1 pour le gris)
8. `origin` : jeu de données d'origine ('MVTec' ou 'RAD')

Nombre lignes dans la table : ?								
* Col	Nom de la colonne	Description	Disponibilité de la variable a priori	Type informatique	Taux de NA	Gestion des NA	Distribution des valeurs	Remarques sur la colonne
		Que représente cette variable en quelques mots ?	Pouvez vous connaître ce champ en amont d'une prédiction ? Avez vous accès à cette variable en environnement de production ?	int64, float etc... Si "object", détaillez.	en %	Quelle mode de (non) - gestion des NA favorisez vous ?	Pour les variables catégorielles comportant moins de 10 catégories, énumérez toutes les catégories. Pour les variables quantitatives, détaillez la distribution (statistiques descriptives de base)	champs libre à renseigner
1	category	pièce à analyser	oui	object (catégorie)	0	N/A	16 modalités	
2	type	découpage du jeu de données en train ou test	non	object (catégorie)	0	N/A	train-test	
3	quality	"good" si la pièce n'a pas de défaut, catégorie de défaut sinon	non	object (catégorie)	0	N/A	53 modalités	Variable cible
4	file	nom du fichier photo, sans extension (.png)	oui	object (variable)	0	N/A	1615 modalités	Il y a 6864 différentes photos mais certains numéros de file sont répétés, 'file' seul n'identifie pas la photo
5	width	largeur de la photo en pixels	oui	int64	0	N/A	900, 1024, 1000, 700, 800, 840, 612	height = width pour chaque photo de MvTech (c'est pourquoi les six premières valeurs sont identiques dans les deux colonnes)
6	height	hauteur de la photo en pixels	oui	int64	0	N/A	900, 1024, 1000, 700, 800, 840, 512	
7	bands	nombre de dimensions pour la couleur (1 pour niveau de gris, 3 pour couleur)	oui	int64	0	N/A	3, 1	Toutes les photos RAD sont en couleur alors que certaines photos de Mvtech sont en grayscale
8	origin	"MVTec" ou "RAD" selon le jeu de données		object (catégorie)	0	N/A	MvTech, RAD	

FIGURE 3.1 – Rapport exploration des données

3.3 Analyses statistiques des meta-données

3.3.1 Répartition des images disponibles

Vue complète On trace un diagramme en barres pour visualiser notre jeu de données complet, en distinguant : la catégorie d'objet à analyser, le type d'image (train/test), la qualité (*good* ou défaut identifié).

Comme on peut le voir dans la figure 3.2, les jeux de données d'images sans anomalie sont de tailles équivalentes, à l'exception de *toothbrush* qui n'a que 72 images "good", contre au moins 200 pour toutes les autres catégories.

On peut voir qu'à l'exception notable du jeu de données RAD (*metal_plate*), on a toujours beaucoup moins d'images avec des anomalies. Il y a toujours plusieurs types d'anomalies, sauf pour *toothbrush* également.

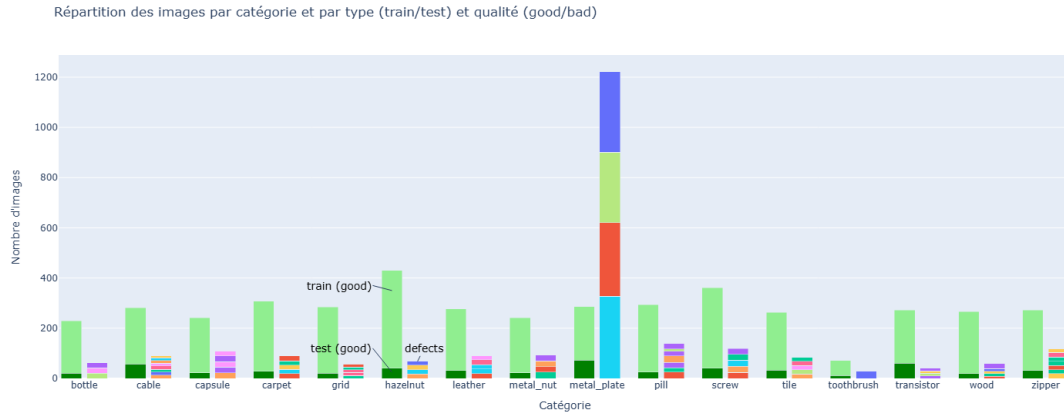


FIGURE 3.2 – Images par catégories, type, qualité

Anomalies par catégorie On peut le confirmer en traçant une boîte à moustache pour voir le nombre d'images avec des anomalies par catégorie, comme dans la figure 3.3. On voit que RAD est une valeur extrême. Pour faciliter la lecture, on a aussi tracé la boîte à moustache pour les images MVTec uniquement.

On voit qu'on a au minimum 30 images avec des anomalies pour chaque catégorie et 75% des catégories ont plus de 60 images avec des anomalies.

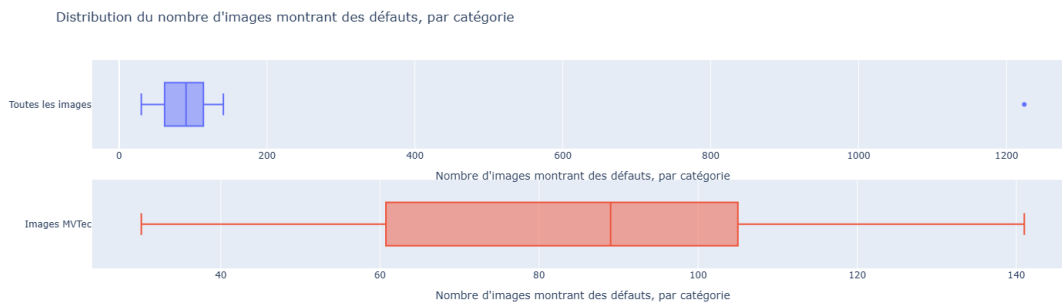


FIGURE 3.3 – Boxplot du nombre d'images d'anomalies par catégorie

Images d'entraînement On peut analyser le nombre d'images disponibles pour l'entraînement pour chaque catégorie. On regarde donc pour chacune des catégories le nombre d'images de type 'train' (puisque le découpage train-test est déjà fait). Par construction, aucune des images d'entraînement ne contient d'anomalies.

On voit dans la figure 3.4 que la répartition est assez homogène. Il y a entre 209 et 320 images d'entraînement par catégorie, sauf pour 2 catégories qui ont des valeurs extrêmes.



FIGURE 3.4 – Boxplot du nombre d'images d'entraînement par catégorie

Couleurs des images On a vérifié que toutes les images d'une catégorie ont toujours les mêmes dimensions et le même nombre de bandes (couleur vs niveaux de gris). On va maintenant regarder les couleurs des images par catégories dans la figure 3.5.

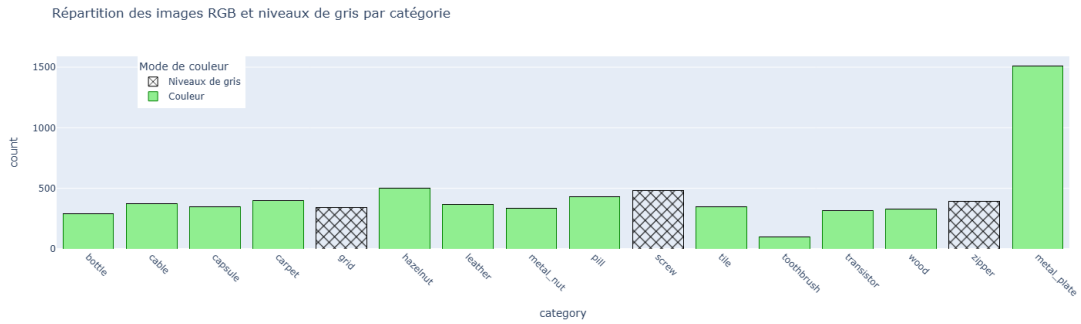


FIGURE 3.5 – Couleur des images par catégories

Dimensions des images On va maintenant analyser le nombre de dimensions à traiter dans les différentes catégories. La taille des images varie et le nombre de bandes également, ce qui va affecter le nombre de paramètres d'entrée pour analyser les images. La répartition est présentée dans la figure 3.6. On voit qu'on a des tailles très variables pour les images, mais la plupart des catégories ont des images avec une définition élevée : 7 catégories avec des images en couleur et 3 avec des images en niveaux de gris en 1024x1024.

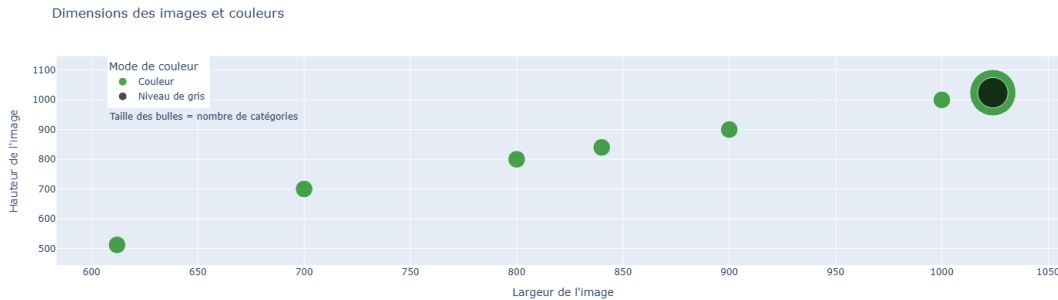


FIGURE 3.6 – Nombre de catégories par dimensions d'images et couleurs

Images des défauts par type de défaut Pour le second niveau d'objectifs, on va vouloir identifier le type de défaut rencontré par une pièce non conforme. Pour cela, on va avoir besoin d'entraîner notre modèle sur les images montrant les défauts. On analyse donc le nombre d'images disponible pour chaque type de défaut à identifier. On pourra ainsi identifier si leur nombre est suffisant ou s'il faut augmenter le nombre d'images.

La distribution est différente pour le jeu de données MVTec et le RAD, comme on le voit dans la figure 3.7.

Distribution du nombre d'images avec des défauts, par type de défaut

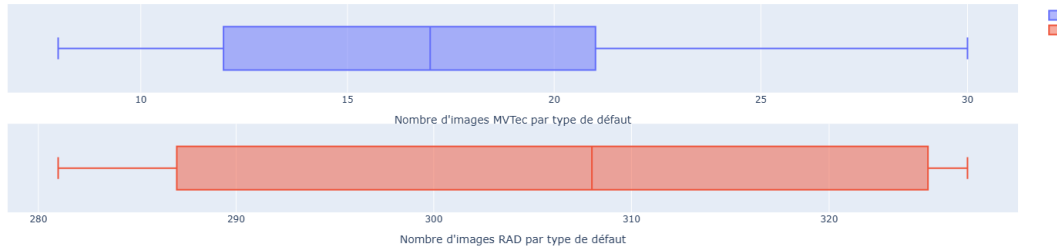


FIGURE 3.7 – Nombre d'images par type de défaut

Le jeu de données RAD est probablement assez complet. En revanche, le jeu de données MVTec va nécessiter des images supplémentaires.

3.4 Augmentation de données

3.4.1 Anomalies pour MVTec

Comme on l'a vu, dans la phase d'analyse de données, il manque des images montrant chaque type de défaut dans le jeu de données MVTec. On a donc développé un script qui génère de nouvelles images à partir des images existantes pour chaque catégorie de défaut.

Ce script prend en paramètres le nombre d'images minimum qu'on attend par type de défaut et par catégorie d'images. Il va parcourir tous les dossiers existants des images et venir ajouter un dossier **augmented** qui contient les images générées ainsi qu'une copie des images initiales. Il procède de la même façon pour les masques pour appliquer les mêmes types de transformations pour que les masques soient toujours applicables aux nouvelles images générées.

Fonctionnement du script

Le script `augmentation_MVTec.py` parcourt tous les dossiers d'images, détermine le nombre d'images augmentées à générer pour chaque image du jeu de données original.

Ensuite, il va générer des images qui correspondent à des transformations aléatoires de l'image originale. On va appliquer aléatoirement plusieurs transformations :

- *flip* : effet miroir horizontal et/ou vertical
- *rotate* : rotation aléatoire de 90, 180 ou 270°
- *gaussian noise* : ajout d'un léger bruit aléatoire sur l'image (non applicable au masque)
- *brightness* : légère modification de la luminosité (non applicable au masque)
- *zoom* : léger effet de zoom et de décalage sur l'image (en gardant les dimensions d'origine de l'image)

Certaines catégories d'images ne se prêtent pas à certaines transformations, en particulier le *flip* ou la rotation. Le script prévoit donc des exceptions pour ces catégories pour restreindre les transformations. L'inconvénient étant que les images augmentées seront beaucoup plus similaires aux images d'origine.

Bibliographie

- BERGMANN, Paul et al. (2019). « MVTEC AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection ». In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. License CC BY-NC-SA 4.0.
- CHENG, Yuqi et al. (2024). « RAD : A Comprehensive Dataset for Benchmarking the Robustness of Image Anomaly Detection ». In : *2024 IEEE 20th International Conference on Automation Science and Engineering*. License CC BY 4.0.